

Цель работы: изучить синтаксис языка Julia на основе русскоязычных источников и сформировать аналитическую подборку примеров его применения.

Аннотированный список источников

1. Белов, Г. В. О возможностях использования языка программирования Julia для решения научных и технических задач / Г. В. Белов, Н. М. Аристова // Вестник Московского государственного технического университета им. Н. Э. Баумана. Серия «Приборостроение».- 2020.- № 4.- Текст : электронный.- URL: <https://cyberleninka.ru/article/n/o-vozmozhnostyah-ispolzovaniya-yazyka-programmirovaniya-julia-dlya-resheniya-nauchnyh-i-tehnicheskikh-zadach> (дата обращения: 19.09.2026).

Аннотация. В статье рассматриваются общие характеристики вычислительной среды и синтаксис языка Julia в сравнении с Python и MATLAB.

2. Решение дифференциальных уравнений в частных производных с использованием функций языка Julia // Вестник науки и образования.- 2021.- № 5 (108).- Текст : электронный.- URL: <https://cyberleninka.ru/article/n/reshenie-differentsialnyh-uravneniy-v-chastnyh-proizvodnyh-s-ispolzovaniem-funktsiy-yazyka-julia> (дата обращения: 19.09.2026).

Аннотация. Описывается работа встроенных алгоритмов языка и проведение агрегирующих вычислений без привлечения тяжеловесных внешних библиотек.

3. Геворкян, М. Н. Возможности языка Julia для обработки статистических данных / М. Н. Геворкян, А. В. Королькова, Д. С. Кулябов // Вестник Российского университета дружбы народов. Серия: Дискретное и непрерывное моделирование и прикладная вычислительная наука. – 2023. – Т. 31, № 1. – С. 5–26. – Текст : электронный. – URL: rudn.ru (дата обращения: 19.09.2026).

4. Julia: функции и структуры-как-функции // Хабр : сообщество программистов.- 2019.- 11 мая.- Текст : электронный.- URL: <https://habr.com/ru/articles/451172/> (дата обращения: 19.09.2026).

Аннотация. Рассматриваются элементы функционального программирования и способы безопасной индексации данных.

5. Белов, Г. В. О возможностях использования языка программирования Julia для решения научных и технических задач / Г. В. Белов, Н. М. Аристова // Вестник МГТУ им. Н. Э. Баумана. Серия «Приборостроение».- 2020.- № 4 (133).- С. 4–19.- Текст : электронный.- URL: <https://cyberleninka.ru/article/n/o-vozmozhnostyah-ispolzovaniya-yazyka-programmirovaniya-julia-dlya-resheniya-nauchnyh-i-tehnicheskikh-zadach> (дата обращения: 19.09.2026).

Аннотация. В работе проводится строгий академический анализ базовых конструкций Julia, его встроенных математических средств, встроенных типов данных и возможностей векторизации.

Примеры решения задач

Алгоритм формирования одномерного числового массива и вычисления среднего арифметического значения средствами встроенных функций Julia.

```
numbers = [10, 20, 30, 40, 50]
total_sum = sum(numbers)
count = length(numbers)
average = total_sum / count
println("Среднее значение равно: ", average)
```

Этот код демонстрирует выполнение базовых арифметических операций над числовыми массивами. Сначала инициализируется одномерный массив элементов, перечисленных через запятую в квадратных скобках. Затем встроенная функция `sum` вычисляет сумму всех элементов, а функция `length` определяет их точное количество. На финальном этапе рассчитывается среднее арифметическое значение путем деления полученной суммы на число элементов, после чего результат выводится на экран. Данный алгоритм показывает, как стандартный набор функций Julia позволяет решать простые вычислительные задачи без использования явных циклов.

При работе со словарями прямое обращение по несуществующему ключу вызывает ошибку. Для безопасного извлечения данных в Julia предусмотрена встроенная функция `get()`. Она возвращает нужное значение, если ключ найден, а

если он отсутствует- подставляет заданный дефолтный результат, предотвращая аварийное завершение программы.

Безопасное извлечение конфигурационных параметров из ассоциативного массива Dict с обработкой исключений KeyError.

```
system_config = Dict("timeout" => 30, "retries" => 3, "output_dir" => "/var/log")
current_timeout = get(system_config, "timeout", 0)
encryption_status = get(system_config, "encryption", "off")
println("Текущий: ", current_timeout)
println("Статус шифрования: ", encryption_status)
```

В листинге показан метод безопасного чтения данных из словаря. Структура Dict хранит информацию в виде пар «ключ- значение» и обеспечивает высокую скорость поиска. Реализованный подход позволяет избежать падения программы, если запрашиваемый ключ отсутствует. При обычной индексации через квадратные скобки обращение к отсутствующему элементу вызывает ошибку KeyError и останавливает выполнение скрипта. Функция get() автоматически проверяет наличие ключа. Если элемент найден, она возвращает его значение, а если нет - подставляет резервный вариант, позволяя программе продолжить работу.

Для предварительного анализа текстовой информации среда разработки предоставляет удобный аппарат регулярных выражений и встроенных строковых функций. Комбинированное использование функции replace() и метасимволов регулярного выражения вида r"..." позволяет быстро очистить текст от знаков препинания, разбить строку на отдельные слова и рассчитать их количественные характеристики.

```
raw_text = "Julia- быстрый язык. Этот язык эффективен, и этот язык удобен."
cleaned_text = lowercase(raw_text)
cleaned_text = replace(cleaned_text, r"[.,—]" => "")
words_list = split(cleaned_text)
println("Список очищенных слов: ", words_list)
```

```
println("Всего слов в тексте: ", length(words_list))  
println("Уникальных слов: ", length(unique(words_list)))
```

Этот фрагмент кода показывает быструю обработку неструктурированного текста. Функция `lowercase` переводит все символы в нижний регистр, а функция `replace` удаляет знаки препинания- это исключает дублирование слов из-за различий в их написании или регистре. На следующем этапе строка разбивается по пробелам на массив отдельных слов с помощью функции `split`. Затем функция `length` подсчитывает количество слов в полученном массиве. Функция `unique` позволяет определить число уникальных лексем.